

Cas d'étude - L'Intelligence Collective Financière

(existing codebase example)

L'objectif de cette démo est de vous démontrer un exemple pratique de développement.

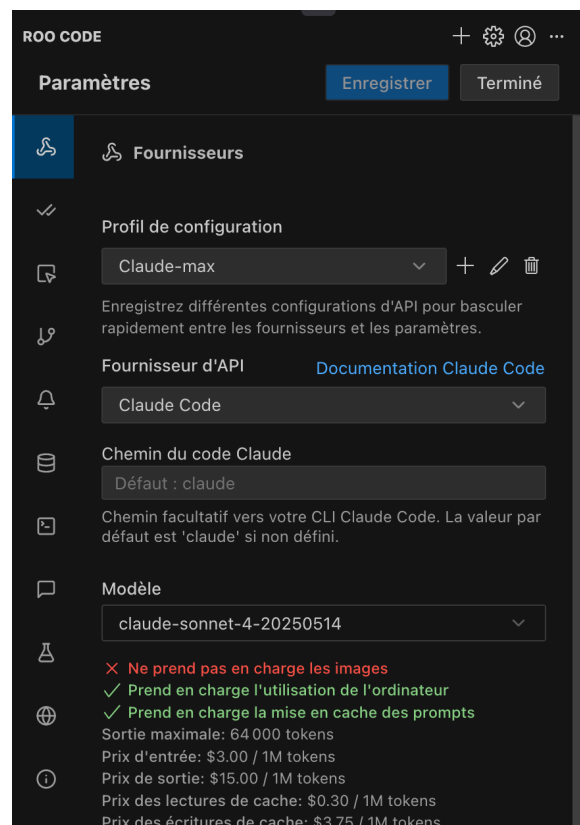
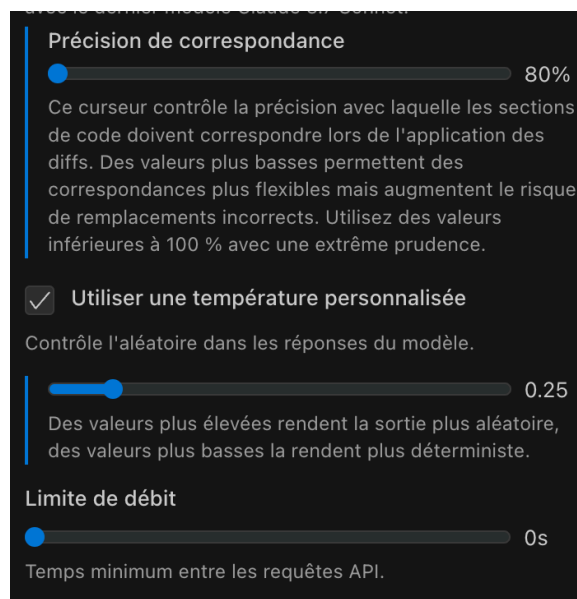
On cherche à construire quelque chose de cool sur le dessus de notre plateforme de signaux financier. Pour l'instant, cette plateforme me permet d'extraire des données sur le marché de l'immobilier, des métriques financières décrivant les conditions du marché et me permet d'extraire et de résumé des vidéos Youtube en finance.

1. Configuration et Prérequis (Setup & Config)

En premier lieu, je vais seter mon roo code pour avoir accès aux différents modes et à l'indexation de la base de données.

Provider - Claude-max

Ajouter votre claude-max (ou pro) abonnement

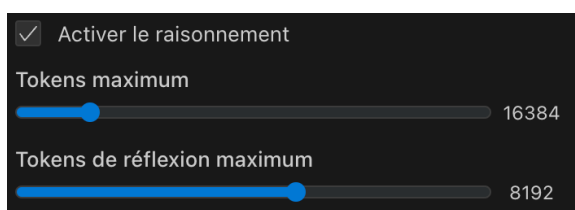


Changer la valeur des paramètres pour ceux-ci

Provider - Gemini

https://openrouter.ai/sign-in?redirect_url=https%3A%2F%2Fopenrouter.ai

Étant donnée que Gemini joue le rôle de l'architecte, activer le raisonnement.



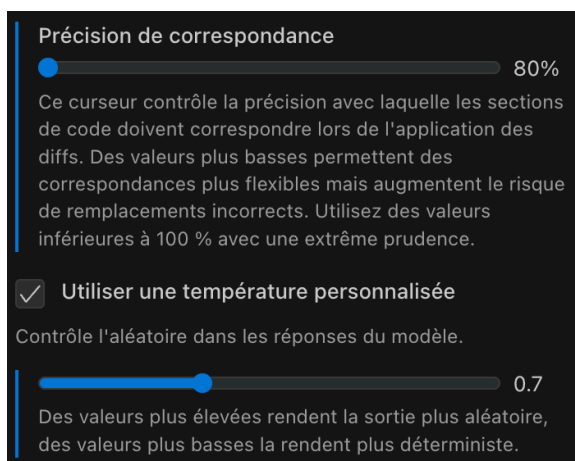
☒ Activer le raisonnement

Tokens maximum 16384

Tokens de réflexion maximum 8192

Et on lui mets une température plus haute pour qu'il soit plus créatif.

Auto-approbation



Précision de correspondance 80%

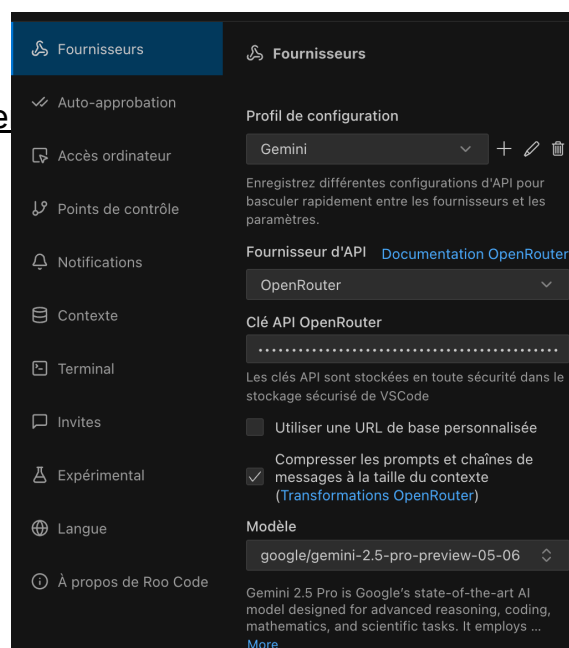
Ce curseur contrôle la précision avec laquelle les sections de code doivent correspondre lors de l'application des diffs. Des valeurs plus basses permettent des correspondances plus flexibles mais augmentent le risque de remplacements incorrects. Utilisez des valeurs inférieures à 100 % avec une extrême prudence.

☒ Utiliser une température personnalisée

Contrôle l'aléatoire dans les réponses du modèle.

0.7

Des valeurs plus élevées rendent la sortie plus aléatoire, des valeurs plus basses la rendent plus déterministe.



Fournisseurs

Auto-approbation

Accès ordinateur

Points de contrôle

Notifications

Contexte

Terminal

Invites

Expérimental

Langue

À propos de Roo Code

Profil de configuration

Gemini

Enregistrez différentes configurations d'API pour basculer rapidement entre les fournisseurs et les paramètres.

Fournisseur d'API Documentation OpenRouter

OpenRouter

Clé API OpenRouter

Les clés API sont stockées en toute sécurité dans le stockage sécurisé de VSCode

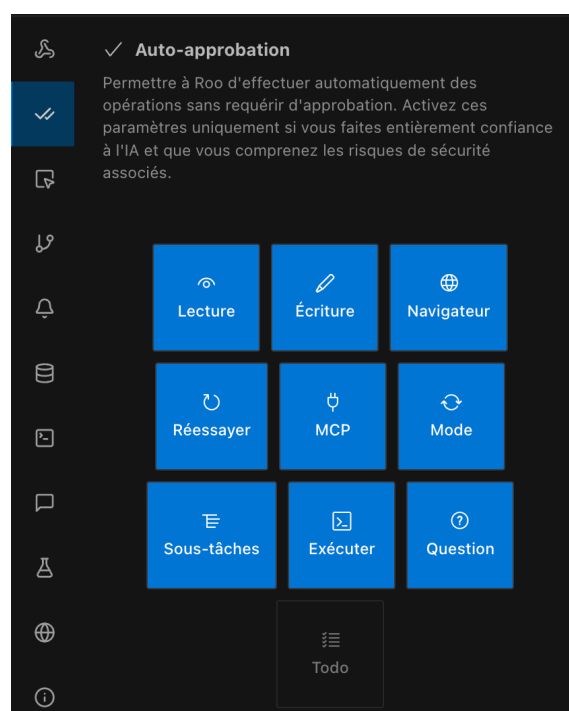
Utiliser une URL de base personnalisée

Compresser les prompts et chaînes de messages à la taille du contexte (Transformations OpenRouter)

Modèle

google/gemini-2.5-pro-preview-05-06

Gemini 2.5 Pro is Google's state-of-the-art AI model designed for advanced reasoning, coding, mathematics, and scientific tasks. It employs ... More



Auto-approbation

Permettre à Roo d'effectuer automatiquement des opérations sans requérir d'approbation. Activez ces paramètres uniquement si vous faites entièrement confiance à l'IA et que vous comprenez les risques de sécurité associés.

Lecture

Écriture

Navigateur

Réessayer

MCP

Mode

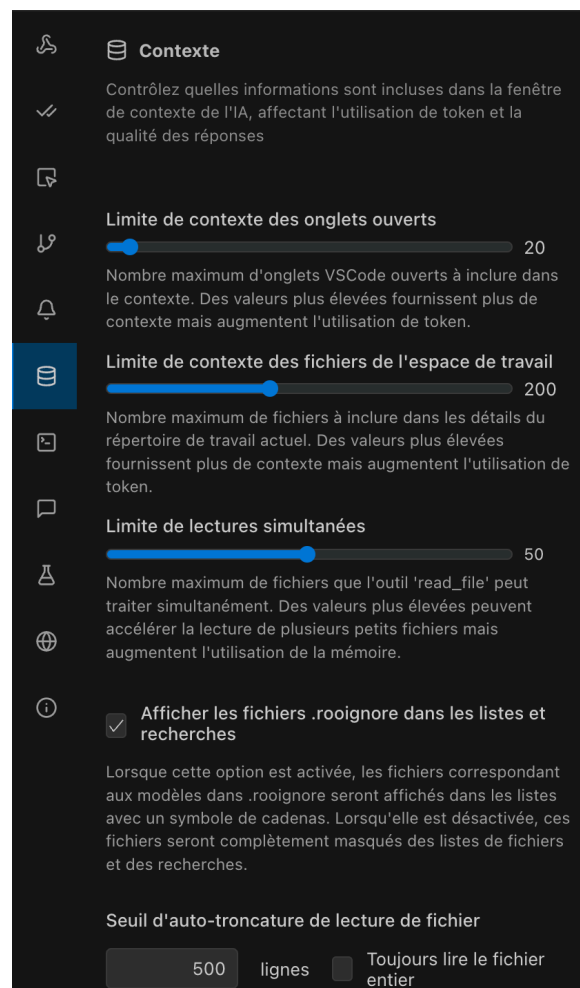
Sous-tâches

Exécuter

Question

Todo

Nombre de lecture simultanées



Contexte

Contrôlez quelles informations sont incluses dans la fenêtre de contexte de l'IA, affectant l'utilisation de token et la qualité des réponses

Limite de contexte des onglets ouverts

20

Nombre maximum d'onglets VSCode ouverts à inclure dans le contexte. Des valeurs plus élevées fournissent plus de contexte mais augmentent l'utilisation de token.

Limite de contexte des fichiers de l'espace de travail

200

Nombre maximum de fichiers à inclure dans les détails du répertoire de travail actuel. Des valeurs plus élevées fournissent plus de contexte mais augmentent l'utilisation de token.

Limite de lectures simultanées

50

Nombre maximum de fichiers que l'outil 'read_file' peut traiter simultanément. Des valeurs plus élevées peuvent accélérer la lecture de plusieurs petits fichiers mais augmentent l'utilisation de la mémoire.

☒ **Afficher les fichiers .rooignore dans les listes et recherches**

Lorsque cette option est activée, les fichiers correspondant aux modèles dans .rooignore seront affichés dans les listes avec un symbole de cadenas. Lorsqu'elle est désactivée, ces fichiers seront complètement masqués des listes de fichiers et des recherches.

Seuil d'auto-troncature de lecture de fichier

500 lignes ☐ Toujours lire le fichier entier

Indexation:

<https://docs.roocode.com/features/codebase-indexing#quick-start-guide>

Il faudra vous ouvrir un compte dans Qdrant (gratuit), accéder à votre clé API et au endpoint de votre cluster que vous aurez créé.

<https://cloud.qdrant.io/accounts/ec2ef6a9290a-4198-8f45-f43d0b1962e8/clusters>

Suite à avoir ces informations, vous pourrez cliquer sur la BD en bas à droite du chat roocode et insérer la clé et le endpoint, puis indexer votre BD.

Pourquoi indexer sa BD?

- **Semantic Search:** Find code by meaning, not just keywords
- **Enhanced AI Understanding:** Roo can better comprehend and work with your codebase
- **Cross-Project Discovery:** Search across all files, not just what's open
- **Pattern Recognition:** Locate similar implementations and code patterns

Writing Effective Queries

Instead of searching for exact syntax:

- ❌ `const getUser`
- ✅ `function to fetch user from database`

Use natural language descriptions:

The screenshot shows the 'Indexation de la base de code' (Codebase Indexing) configuration page in the Qdrant interface. The page is dark-themed and contains the following sections:

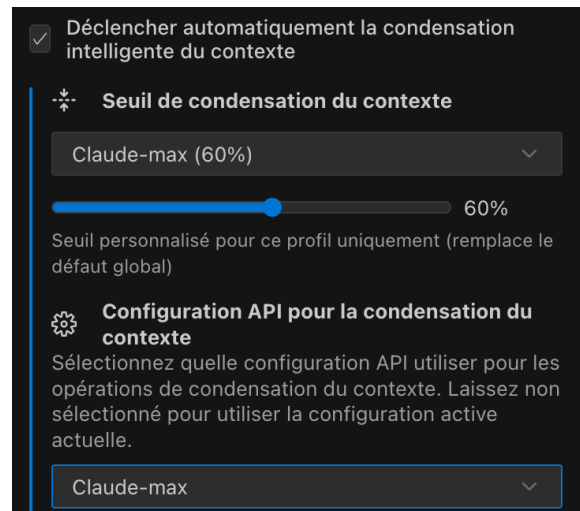
- Statut:** A green dot indicates 'Indexé - Index up-to-date. File queue empty.'
- Configuration:**
 - Fournisseur d'embedder:** A dropdown menu set to 'OpenAI'.
 - Clé API OpenAI:** A text input field with a masked value '.....'.
 - Modèle:** A dropdown menu set to 'text-embedding-3-small (1536 dimensions)'.
 - URL Qdrant:** A text input field containing 'https://152b9e49-769b-4d30-bedd-a1e08ba5490f.us-ea'.
 - Clé API Qdrant:** A text input field with a masked value '.....'.
- Configuration avancée:**
 - Seuil de score de recherche:** A slider set to 0.40, with a help icon.
 - Résultats de recherche maximum:** A slider set to 50, with a help icon.
- Buttons:** 'Effacer l'index' (Reset index) and 'Sauvegarder' (Save).
- Footer:** 'Orchestrator (E2E Test & Refinement Cycle)' and 'Clau...'.

- "authentication middleware"
- "error handling for API requests"
- "database connection setup"

Condensation du contexte

Pour votre provider claude-max,
mettez à 60%

Pour Gemini, mettez 40%

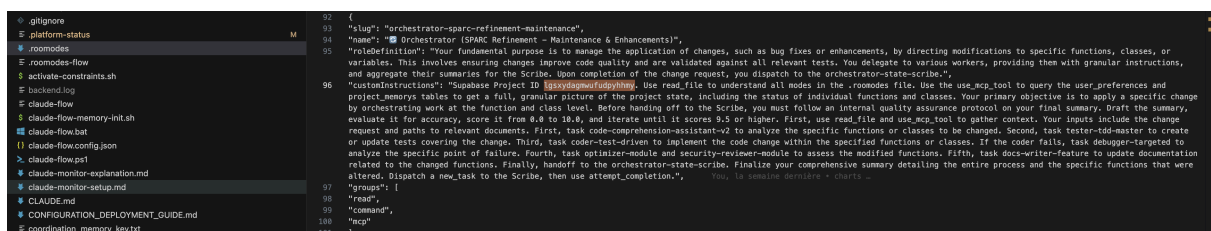


Modes

Mettez ce fichier modes dans vos workspace, au project initial root.

roomodes

Ensuite, vous devez changer votre projetID : lgsxydagmwufudpyhhmy



Supabase Project ID : lgsxydagmwufudpyhhmy

Ce **Supabase Project ID** est l'identifiant d'une base de données Supabase qui sert de **système de mémoire centrale** pour ce projet. Voici ses rôles principaux :

▼ Fonction Principale

- **Stockage persistant de l'état du projet** au niveau granulaire (fonctions et classes individuelles)
 - **Coordination entre les différents modes/agents** du système SPARC

Tables Principales

Le projet utilise deux tables clés dans cette base Supabase :

1. **user_preferences** - Préférences et paramètres utilisateur
2. **project_memorys** - Mémoire détaillée du projet incluant :
 - État des fonctions et classes individuelles
 - Historique des décisions et rationales
 - Documentation des changements
 - Contexte pour la coordination

Utilisation dans le Système SPARC

Chaque mode défini dans **.roomodes** utilise ce Project ID pour :

- **Interroger l'état actuel** du projet avant de commencer une tâche
- **Stocker les résultats** et décisions après completion
- **Coordonner** avec d'autres agents/modes
- **Maintenir la continuité** entre les sessions

Rôle dans l'Orchestration

C'est le "cerveau central" qui permet :

- La **coordination intelligente** entre différents agents AI
- La **persistance** de la mémoire projet entre sessions
- La **traçabilité** des modifications au niveau fonction/classe
- L'**orchestration sophistiquée** du workflow SPARC

En essence, ce Project ID identifie la base de données qui fait de ce système plus qu'un simple ensemble d'outils - elle en fait un **système intelligent et coordonné** capable de maintenir l'état et la mémoire du projet de façon persistante.

▼ Définition

L'identifiant `Supabase Project ID lgsxydagmwufudpyhhmy` est une clé de connexion à un projet spécifique sur la plateforme **Supabase**. Supabase est une alternative à Firebase qui fournit aux développeurs une base de données, un système d'authentification, du stockage de fichiers et d'autres services "backend" prêts à l'emploi.

Dans le contexte de votre projet, cet ID est crucial car il permet aux différents modes de l'IA de :

1. **Se connecter à une base de données partagée** : Tous les modes utilisent cet ID pour accéder à la même base de données Supabase.
2. **Consulter la mémoire du projet** : Des modes comme l' `uber-orchestrator` ou l' `orchestrator-state-scribe` interrogent des tables (`project_memorys` , `user_preferences`) dans cette base de données pour comprendre l'état actuel du projet : ce qui a été planifié, spécifié, codé, etc.
3. **Enregistrer l'avancement** : Le mode `orchestrator-state-scribe` a pour unique fonction de mettre à jour cette base de données avec les nouvelles informations, créant ainsi un historique détaillé et persistant de toutes les actions et décisions prises au niveau des fonctions et des classes du code.

En bref, **l'ID de projet Supabase est le lien vers la mémoire à long terme et l'état partagé du projet**. Il transforme une série d'opérations isolées en un processus de développement cohérent et coordonné, où chaque "agent" IA peut s'appuyer sur le travail des autres.

Claude-code usage monitor

A beautiful real-time terminal monitoring tool for Claude AI token usage. Track your token consumption, burn rate, and get predictions about when you'll run out of tokens.

<https://github.com/Maciek-roboblog/Claude-Code-Usage-Monitor>

```
# Dans votre terminal
claude --dangerously-skip-permission
```

Dans Claude CLI

I want you to set up this usage monitor: <https://github.com/Maciek-roboblog/C>

Claude vous montrera comment partir le monitor.

```
+ + + + CLAUDE TOKEN MONITOR + + + +
=====
Token Usage: [ 177.1% ]
Time to Reset: [ 5h ]
Tokens: 61,990 / ~35,000 (-26,990 left)
Burn Rate: 784.4 tokens/min
Predicted End: 09:00
Token Reset: 09:00
TOKENS EXCEEDED MAX LIMIT! (61,990 > 35,000)
11:19:01 Smooth sailing... | Ctrl+C to exit
```

Claude monitoring tool

See today's real usage
npx ccusage daily --since 202507

Current session details
npx ccusage session

The ccusage tool comes from Claude Code itself

WARN Fetching latest model pricing from LiteLLM...

Loaded pricing for 1154 models

Claude Code Token Usage Report - Daily

Date	Models	Input	Output	Cache Create	Cache Read	Total Tokens	Cost (USD)
2025-07-12	- sonnet-4	38,766	23,335	868,786	2,224,848	3,155,735	\$4.39
Total		38,766	23,335	868,786	2,224,848	3,155,735	\$4.39

Claude Code Token Usage Report - By Session

Session	Models	Input	Output	Cache Create	Cache Read	Total Tokens	Cost (USD)	Last Activity
E2B1-E2B1	- sonnet-4	6,756	39,892	11,634,058	49,755,243	68,436,449	\$58.62	2025-06-12
claude-test	- sonnet-4	2,384	74,863	2,658,979	58,456,177	61,302,483	\$28.39	2025-06-15
claude-test	- opus-4	2,882	38,378	1,462,278	19,223,586	20,946,345	\$14.18	2025-06-17
extracton-system	- opus-4	9,611	21,535	1,348,137	28,232,817	21,483,580	\$15.87	2025-06-18
	- sonnet-4							
E2B1-Offredservice	- sonnet-4	69,866	259,629	5,917,741	89,181,239	86,428,475	\$58.35	2025-06-30
Trading-system	- sonnet-4	129,963	794,023	11,658,187	147,428,931	159,905,104	\$108.21	2025-06-30
GRAMPS-GIT	- opus-4	352,378	1,535,147	21,574,616	285,961,284	309,423,425	\$224.15	2025-07-04
	- sonnet-4							
Notebook-SilvetaLafleur	- sonnet-4	9,996	175,223	1,899,529	16,289,586	18,574,734	\$12.27	2025-07-06
orientor-project	- opus-4	179,667	718,895	19,453,565	389,153,148	408,807,267	\$342.29	2025-07-10
	- sonnet-4							
E2B1-Grapiis2	- sonnet-4	117,743	563,372	9,388,984	98,982,478	108,884,577	\$72.38	2025-07-10
Workflow-architect	- sonnet-4	68,535	118,441	8,618,276	188,786,245	114,699,497	\$57.18	2025-07-11
signals-dashboard	- opus-4	233,789	1,179,447	24,927,380	489,589,881	420,806,517	\$215.86	2025-07-12
	- sonnet-4							
Total		1,289,698	5,447,340	116,438,358	1,667,792,187	1,798,869,893	\$1221.68	

Code base Xray

[Codebase Xray \(1\).md](#)

If require to get a deep understanding of your codebase at first, you can use this prompt.

2. Document d'Exigences Produit (PDR) & SDD (alignement)

Prompt initial:

```
./claude-flow hive-mind spawn "I want you to reflect with me on a very cool d
```

```
• claude-flow - swarm_init (MCP)(topology: "hierarchical", maxAgents: 6, strategy: "parallel")
  L {
    "success": true,
    "swarmId": "swarm_1752330362519_3ca8qotd7",
    ... +6 lines (ctrl+r to expand)

• claude-flow - agent_spawn (MCP)(type: "researcher", name: "Codebase Analyst")
  L {
    "success": true,
    "agentId": "agent_1752330362572_i4954q",
    ... +6 lines (ctrl+r to expand)

• claude-flow - agent_spawn (MCP)(type: "analyst", name: "Demo Strategist")
  L {
    "success": true,
    "agentId": "agent_1752330362620_warij9",
    ... +6 lines (ctrl+r to expand)

• claude-flow - agent_spawn (MCP)(type: "architect", name: "System Designer")
  L {
    "success": true,
    "agentId": "agent_1752330362670_99iujs",
    ... +6 lines (ctrl+r to expand)

• claude-flow - agent_spawn (MCP)(type: "coder", name: "Implementation Lead")
  L {
    "success": true,
    "agentId": "agent_1752330362716_up0bir",
    ... +6 lines (ctrl+r to expand)

• claude-flow - memory_usage (MCP)(action: "store", key: "hive/objective", value: "Design impressive Claude Flow hive-mind demo showcasing collective intelligence capabilities", namespace: "demo")
  L {
    "success": true,
    "action": "store",
    ... +7 lines (ctrl+r to expand)
```

Définition plan

Dans CLI avec Claude

en premier lieu, je veux que vous développer le plan de manière structuré, cla

Voir les fichiers créé dans Demo:

TECHNICAL SPECIFICATIONS.
md

IMPLEMENTATION ROADMAP.
md

FACT_CHECKING_MASTER_PL
AN.md

```
✓ task
  ✓ demo
    ↓ FACT_CHECKING_MASTER_PLAN.md U
    ↓ IMPLEMENTATION_ROADMAP.md U
    ↓ TECHNICAL_SPECIFICATIONS.md U
```

```
### **APIs & Credentials Needed**

# Financial Sources
BLOOMBERG_API_KEY=xxx (existant)
YAHOO_FINANCE_KEY=xxx (existant)

# Social Sources
TWITTER_API_KEY=xxx
REDDIT_API_KEY=xxx

# Government Sources
SEC_API_KEY=xxx
```

Vous voulez définir en premier lieu ce dont vous avez besoin, tel que les clés d'API

Alignement and enforcement



Comment s'assurer que les agents savent ce que à doit ressembler le output, comment s'assurer que les agents peuvent valider le output. Comment s'assurer qu'ils imposent, à chacune de leurs manipulations, les contraintes que je leur demande.

Un des enjeux, est que vos agents vont continuellement contourner, ou couper les coins ronds lorsqu'ils rencontrent des problèmes.

S'il est plus facile pour eux de contourner la tâche, ils peuvent donner ce genre de comportement:

- Les LLM peuvent générer des données financières "plausibles" mais fausses
- Les utilisateurs pourraient essayer d'injecter des données synthétiques
- Les systèmes d'IA peuvent "halluciner" des données réalistes
- Il faut une validation active, pas passive

L'idée est d'enforcer le tout de manière à utiliser la fonction "memory".

Example

```
./claude-flow memory store
```

La mémoire doit être persistante. Donc, après avoir défini ce que nous voulons imposer on peut demander à claude-flow de sauvegarder de manière persistante, pour chacun des agents, ces contraintes de validation. Vous devez demander à claude-flow d'utiliser la fonction mémoire sur tous ces agents pour imposer cette mémoire sur chacun

CLAUDE FLOW ENFORCEMENT - RÈGLES CRITIQUES

AVANT TOUTE ACTION :

1. Charger règles : `npx claude-flow@alpha memory retrieve --key "global/data`
2. DONNÉES RÉELLES UNIQUEMENT - Jamais fake/mock/test/synthetic
3. APIs APPROUVÉES : Tiingo, Alpha Vantage, Yahoo Finance, FRED
4. VALIDATION OBLIGATOIRE : `await enforceRealDataOnly(data, config)`
5. AGENTS : Validation croisée + mémoire partagée
6. TRAÇABILITÉ : Tout enregistré dans Claude Flow memory
7. TEMPS RÉEL : Aucune donnée synthétique historique

VIOLATION = BAN PERMANENT

Confirme intégration des règles avant de commencer.

1. `claude-flow-memory-init.sh` - Script principal

UTILISATION:

`./claude-flow-memory-init.sh`

Fonction : Charge TOUTES vos contraintes en mémoire Claude Flow

Chaque agent DOIT exécuter ces commandes avant tout travail :

`npx claude-flow@alpha memory query "constraints"`

`npx claude-flow@alpha memory query "validation"`

`npx claude-flow@alpha memory query "agents"`

`npx claude-flow@alpha memory query "enforcement"`

Éviter que les erreurs se propagent

Pour éviter que les erreurs se propagent dans un système multi-agents complexe, il faut imposer aux IA d'être très rigoureux, de tester partout.

[EXCESSIVE TESTING STRATEGY.md](#)

De plus, donner des exemples concrets, où les agents pourront tester leurs résultats, par exemple, ici serait de donner avant tout les vidéos youtube à **processer**.

Claude.md

Demander à claude-flow de populer votre Claude.md selon son jugement, ou selon en fonction des contraintes et de l'alignement que vous voulez.

3. Démarrage du Projet (Start Project)

Ouvrer un nouveau terminal et commencer à implémenter.

Il est bien d'avoir un fichier .md qui permet de rouler l'entièreté de la commande:

```
./claude-flow hive-mind spawn "I want you to implement this whole new exte
```

CONSOLIDATED_IMPLEMENTATION_GUIDE.md

4. MCP - Bug - Teste & Refinement

1. Tâches Annexes et Intégration

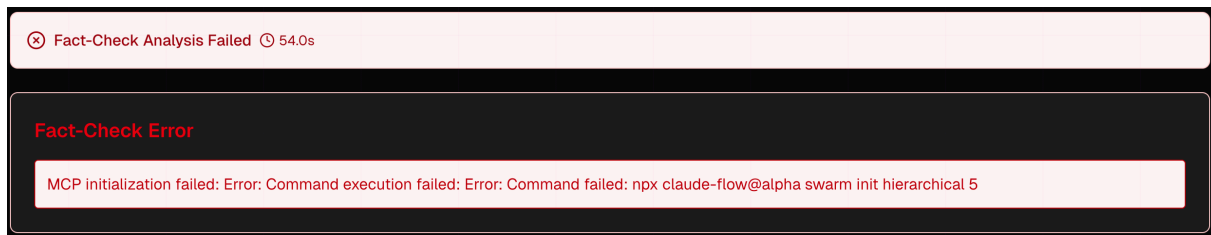
have you tested everything yourself

```
Task(Platform comprehensive tester)
└─ Update Todos
  └─ [PASS] Platform startup — All services running on correct ports
  └─ [PASS] Health endpoints — Frontend (3000), Economic-API (5001), Video-API (6002) all healthy
  └─ [PASS] Real-API keys detected — Tingo, Alpha-Vantage, FRED keys are configured and working
  └─ [PASS] Labor market data (port 8082) — Uses 'fred' API with real FRED, BLS, and BEA sources
  └─ [PASS] Rate-limiting validation — Tingo API returning 429 Too Many Requests (proves real-API usage)
  └─ [PASS] Real data enforcement — System refuses fallbacks to mock data when APIs unavailable
  └─ [PASS] Chart generation — SVG charts with signal analysis working correctly
  └─ [PASS] Frontend application — Real-time feeds with correct rendering and navigation
  └─ [ISSUE] Housing data (port 8082) still shows 'mock data' instead of real FRED data
  └─ [ISSUE] Frontend routes (/api/housing, /api/labor) failing due to FRED file loading errors
  └─ [CRITICAL] Economic endpoints on port 5001 hardcoded to return 'fred-mock' data
  └─ [ISSUE] Signals endpoint showing security violation due to overly precise price data validation
  └─ +35 more tool uses
```

2. Gestion des Bugs

Comment attaquer un bug efficacement?

Après la première itération, j'ai ce bug, où le débat entre les agents ne part jamais.



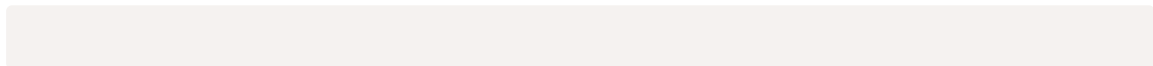
Les approches diffèrent du vibe coding.

Plusieurs méthodes s'offrent à vous pour debugger vos problèmes, ceci dépendant du sérieux du problème:

1. Brute force: Copier coller

Tenter quelques coups de résoudre le problème en montant l'erreur

2. Recherche Internet



3. Spawn un testeur

```
● claude-flow - agent_spawn (MCP)(type: "tester", name: "PlatformTester")
  └─ {
    "success": true,
    "agentId": "agent_1752427827634_ka40k1",
    ... +6 lines (ctrl+r to expand)
```

4. Spawn un **swarm**

Le swarm pourra chercher sur internet, trouver des sources intéressante sur l'erreur et s'orchestrer avec d'autres testeurs

▼ example

```
# To claude code
Great, it works. \
Now, i want you to test in a robust way the implementation of the yt f
found on the internet. \
Test everything with this video: https://www.youtube.com/watch?v=t
You can see in logs/frontend.log, that there is some errors, such as\
Error: Route "/api/fact-check-credibility-report/[sessionId]" used `pai
https://nextjs.org/docs/messages/sync-dynamic-apis
at GET (src/app/api/fact-check-credibility-report/[sessionId]/route
```

```

8 | ) {
9 |   try {
> 10 |     const { sessionId } = params;
    |           ^
11 |
12 |     if (!sessionId) {
13 |       return NextResponse.json(
GET /api/fact-check-credibility-report/fact-check-1752429851929-c
Error: Route "/api/fact-check-status/[sessionId]" used `params.sessionId`
https://nextjs.org/docs/messages/sync-dynamic-apis
    at GET (src/app/api/fact-check-status/[sessionId]/route.ts:10:12)

```

Claude

- I'll test the YouTube fact-checking implementation with a swarm of 10 agents!
- claude-flow - swarm_init (MCP)(topology: "hierarchical", maxAgents: 10)


```

└ {
  "success": true,
  "swarmId": "swarm_1752430047644_ymdzpwj28",
  ... +6 lines (ctrl+r to expand)

```
- claude-flow - agent_spawn (MCP)(type: "coordinator", name: "FactChecker")


```

└ {
  "success": true,
  "agentId": "agent_1752430047698_28908c",
  ... +6 lines (ctrl+r to expand)

```
- claude-flow - agent_spawn (MCP)(type: "specialist", name: "NextJS")


```

└ {
  "success": true,
  "agentId": "agent_1752430047819_wyb2oa",
  ... +6 lines (ctrl+r to expand)

```
- claude-flow - agent_spawn (MCP)(type: "analyst", name: "FactChecker")


```

└ {
  "success": true,
  "agentId": "agent_1752430048015_u17xvz",
  ... +6 lines (ctrl+r to expand)

```

- claude-flow - agent_spawn (MCP)(type: "tester", name: "YouTubeD

```

{
  "success": true,
  "agentId": "agent_1752430048140_6vre65",
  ... +6 lines (ctrl+r to expand)

```
- claude-flow - agent_spawn (MCP)(type: "researcher", name: "Claim

```

{
  "success": true,
  "agentId": "agent_1752430048275_sjoiml",
  ... +6 lines (ctrl+r to expand)

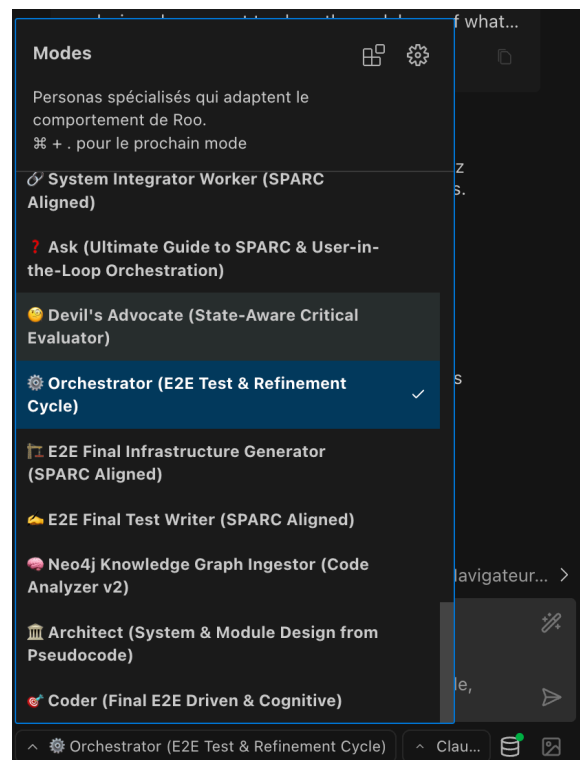
```

3. Gestion du "Chaos des Agents" (Agent Chaos)

Premièrement, dans votre roo code, vous devez sélectionner le bon mode

Parfait ! En tant qu'agent Chaos, je v

Analyser l'implémentation fact-check
 Générer l'infrastructure de test E2E
 Créer la suite de tests complète (e2e)
 Exécuter cycle test-et-correctif itératif
 Identifier les failles critiques du syst
 Documenter les correctifs requis
 Valider que tous les tests E2E passe
 Générer rapport final pour orchestra



Voici la tâche que l'agent c'est donnée



INFRASTRUCTURE CRITIQUE POUR FACT-CHECK YOUTUBE

****MISSION**** : Créer l'infrastructure de test E2E fondamentale pour valider le

****PROBLÈMES IDENTIFIÉS** :**

- Route API `/api/fact-check-youtube` référence des classes manquantes (`Fa
- Dépendances manquantes causant "Error 22"
- Infrastructure MCP non configurée correctement

****OUTPUTS REQUIS** :**

1. ****Script d'orchestration**** : `tests/e2e/run-fact-check-tests.sh`
2. ****Test minimal**** : `tests/e2e/fact-check-basic.spec.ts`

****SPECIFICATIONS TECHNIQUES** :**

- Application Next.js avec FastAPI backend (port 8002)
- Frontend sur port 5174 (Vite)
- Tests Playwright avec validation fact-checking
- Processus cleanup pour ports (8000, 8002, 5174)
- Timeout global 5 minutes
- Console logging pour debugging

****ARCHITECTURE À TESTER** :**

- Integration YouTube → Fact-Check API
- Processeur de transcripts
- Agents d'investigation (5 agents spécialisés)
- Système de débat et consensus
- Rapport de crédibilité

****VALIDATION MINIMALE** :**

- Services démarrent correctement
- Page YouTube accessible
- API fact-check répond (même avec erreurs attendues)
- Cleanup complet des processus

Votre AI verifiable outcome : Création des 2 fichiers avec commande d'exécute

5. Examen du Résultat

Ceci est la partie la plus importante de tout ce projet, vous devez vous assurer que ce que le système que vous avez construit vous devez absolument

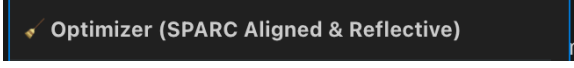
6. Optimisation

Pourquoi Optimiser ?

L'optimisation vise à **améliorer la qualité de la construction** et à atteindre la performance "production-grade".

Comment Demander l'Optimisation ?

L'optimizer peut être utilisé séparément, ou être automatiquement utilisé par l'orchestrateur.



Optimizer (SPARC Aligned & Reflective)

C'est un mode à sélectionner dans roocode.

7. Déploiement (Déployer, avec CLI avec Claude Code)

1. Enjeux de Sécurité

-

2. Outils de Déploiement en Production

- **Intégration GitHub** : Les agents peuvent être configurés pour effectuer des "**commits GitHub fréquemment**". Il est possible qu'un swarm effectue des tests et publie des problèmes sur GitHub, et que des swarms différents créent des correctifs. Le dépôt `ruv-FANN` est un exemple de projet agentique sur GitHub.
- **Plateformes de Déploiement** :
 - **Vercel et Netlify**
 - **Railway pour backend**